Aufgabenblatt 3

Übung zu Einführung in die Bildverarbeitung

Hanna Beitz, Malin Haas, Lasse Huber-Saffer, Tim Radke,
Teodora Rogozenco, Jannis Scheff, Christian Wilms

SoSe 2026

Ausgabe: 1. Mai 2026 - **Abgabe bis: 8. Mai 2026, 10:00 per Moodle!**

Lernziele

- Zusammenhang zwischen Mittelwert und Varianz verstehen
- Mit Operationen auf Bildern sinnvolle Aufgaben lösen
- Unterscheidung von räumlichen Operationen vertiefen
- Einfluss der Überlaufbehandlung, v.a. re-scaling, auf das Ergebnis verstehen
- Effizientes Arbeiten mit Numpy-Arrays ohne Schleifen

Aufgabe 1 — Varianz in einem Durchlauf - 20 Punkte - Theorieaufgabe
Wie in der Vorlesung bereits angedeutet lässt sich die Varianz eines Bildes

$$\sigma^2 = \frac{1}{N_{\text{cols}} N_{\text{rows}}} \sum_{x=1}^{N_{\text{cols}}} \sum_{y=1}^{N_{\text{rows}}} [I(x,y) - \mu_I]^2 \tag{1}$$

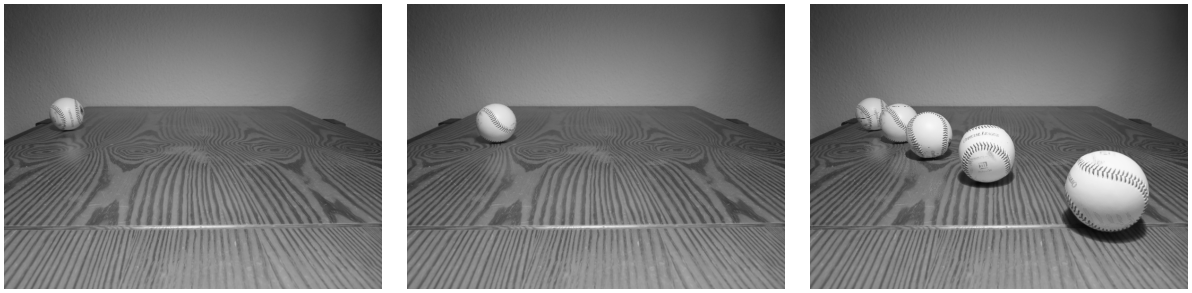
auch mit nur einem Durchlauf durch das Bild berechnen:

$$\sigma^2 = \left[\frac{1}{N_{\text{cols}} N_{\text{rows}}} \sum_{x=1}^{N_{\text{cols}}} \sum_{y=1}^{N_{\text{rows}}} I(x,y)^2 \right] - \mu_I^2. \tag{2}$$

Die Definition des Mittelwerts μ_I eines Bildes I könnt ihr den Folien entnehmen. Ein Durchlauf bedeutet dabei, dass jedes Pixel nur einmal betrachtet wird. In Formel 2 ist dies daran zu erkennen, dass der Mittelwert μ_I nicht mehr in jedem Summanden benötigt wird, sondern nur noch einmal am Ende abgezogen wird. Das heißt, dass der Mittelwert im selben Durchlauf parallel zur Doppelsumme in der Varianz berechnet werden kann.
Beweist nun, dass die beiden Formeln 1 und 2 zur Berechnung der Varianz gleich sind. Startet dazu bei der Formel 1 und zerlegt den Summanden der Doppelsumme. Überlegt anschließend, was die drei einzelnen Teile bedeuten und wie sie vereinfacht werden können.
Tipp: Notiert nach der initialen Zerlegung des Summanden die Formel mit drei Doppelsummen, um die einzelnen Teile einfacher zu untersuchen.

Aufgabe 2 — Bildinhalte kombinieren - 10+12+10+8(+20)=40(+20) Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `matplotlib`
In der Vorlesung wurde das Mitteln über mehrere zeitlich versetzt aufgenommene Bilder einer identischen Szene als Möglichkeit zur Unterdrückung des Rauschens präsentiert. Diese Idee soll in dieser Aufgabe genutzt werden, um aus Einzelbildern, die verschiedene Zustände einer Szene zeigen, ein Gesamtbild zu erzeugen. Ein Beispiel dazu bietet das kombinierte Serienbild in Abbildung 1, das aus mehreren Einzelbildern besteht, die jeweils einen Bewegungszustand des Balls darstellen. Konkret geht es in dieser Aufgabe um die vier Einzelbilder des `kombibild.zip` aus dem Moodle, die zu einem Bild zusammengesetzt werden sollen.

1. Ladet die vier Einzelbilder des `kombibild.zip` aus dem Moodle in Python. Zunächst muss aus den vier Einzelbildern ein möglichst perfektes Hintergrundbild ohne Kronkorken erzeugt werden. Wenn man die Kronkorken als Rauschen annimmt, kann man nach dem Gesetz der großen Zahlen durch Mittelung über die Bilder das Rauschen, bzw. hier die Kronkorken, zumindest recht gut entfernen. Reichen vier Bilder dafür aus? Probiert es aus!
2. Überprüft das Ergebnis visuell und überlegt euch eine bessere aber sehr ähnliche Möglichkeit für das Verknüpfen der vier Pixel an jeweils einer Koordinate. Betrachtet dabei keine anderen Koordinaten wie etwa die der räumlichen Nachbarn! Das Ergebnis sollte nahezu perfekt aussehen.
3. Nutzt nun euer erzeugtes Hintergrundbild, um die veränderten Pixel in jedem der vier Einzelbilder zu ermitteln. Damit ihr wirkliche Veränderungen vom Grundrauschen unterscheiden könnt, empfiehlt sich die Nutzung eines Schwellenwerts für die Mindeststärke der Veränderung. Den Schwellenwert müsst ihr selbst ermitteln.
4. Ersetzt nacheinander für jedes der vier Bilder die veränderten Pixel im Hintergrundbild durch die entsprechenden Pixel des Einzelbildes. D.h. die Pixel, die sich zwischen dem Hintergrundbild und dem ersten Einzelbild verändert haben, werden im Hintergrundbild durch die entsprechenden Pixel des ersten Einzelbilds ersetzt usw. Speichert das Ergebnisbild ab. Welche Figur ergeben die zusammengesetzten Kronkorken nun?
5. **Zusatzaufgabe:** Nehmt selbst Einzelfotos oder ein Serienfoto auf und versucht euer Verfahren darauf anzuwenden. Achtet dabei auf eine möglichst ruhige Kamera und benutzt wenn möglich ein Stativ. Um ein Farbbild in ein Graustufenbild umzuwandeln, könnt ihr die Funktion `skimage.color.rgb2gray` nutzen und das Ergebnis mit 255 multiplizieren. Originelle Ergebnisse präsentieren wir gerne in der Übung!



(a) (b) (c)

Abbildung 1: Die ersten beiden Einzelbilder (a und b) eines kombinierten Serienbildes (c), das aus der Kombination von fünf Einzelbildern besteht.

Aufgabe 3 — Operationen auf Bildern - 4+4+4+4+4=20 Punkte - Theorieaufgabe

Schaut euch die fünf folgenden Python-Funktionen an und beantwortet zu jeder Funktion, ob es sich dabei um eine Punktoperation, eine Nachbarschaftsoperation, eine geometrische Transformation oder eine globale Operation handelt. Gebt jeweils eine kurze Begründung. Nehmt an, dass das Bild eine Auflösung von $n \times n$ hat. Die Ränder der Bilder werden tlw. nicht gesondert behandelt, um den Code auf das Wesentliche zu reduzieren. Ignoriert daher das Verhalten der Funktionen an den Bildrändern.

```
def a(img):
    return img[::-1,::-1]

def b(img):
    result = np.zeros_like(img, dtype=np.int)
    for x in range(img.shape[0]):
        for y in range(img.shape[1]):
            result[x,y] = np.mean(img[:,y])
    return result

def c(img):
    return np.maximum(img, 255 - img)
```

```
def d(img):
    result = np.copy(img)
    for x in range(img.shape[0]-1):
        for y in range(img.shape[1]-1):
            result[x,y] = np.sum(img[:x+1,:y+1])
    return result

def e(img):
    return img[1:,1:]+img[:-1,:-1]+img[1:,:-1]+img[:-1,1:]
```

Aufgabe 4 — Interpretation von Satellitenbildern - 15+5=20 Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.util`, `matplotlib`
In dieser Aufgabe ist die Nutzung von Schleifen nicht erlaubt!
In der Fernerkundung werden verschiedene Indizes genutzt, um zu prüfen, welche Pixel eines Satellitenbildes etwa grüne Vegetation oder Wasser zeigen. Ein Beispiel ist der Vegetationsindex NDVI (Normalized Difference Vegetation I¸ndex), der für jedes Pixel eines Satellitenbildes berechnet wird. Der NDVI ist ein Indikator für lebende, grüne Vegetation. Für seine Berechnung werden die Intensitäten des roten Lichts (Red) und des nahen Infrarots (NIR) in einem Pixel benötigt. Mithilfe dieser Werte kann der NDVI für ein Pixel als

$$\mathrm{NDVI} = \frac{(\mathrm{NIR} - \mathrm{Red})}{(\mathrm{NIR} + \mathrm{Red})}$$

berechnet werden. Ähnlich ist der Wasserindex NDWI (Normalized Difference Water Index) definiert. Dieser ist ein Indikator für Wasser und basiert auf den Intensitäten des nahen Infrarots und des kurzwelligen Infrarots (SWIR):

$$\mathrm{NDWI} = \frac{(\mathrm{NIR} - \mathrm{SWIR})}{(\mathrm{NIR} + \mathrm{SWIR})}.$$

Für die Erkennung wurde Schnee der Normalized Difference Snow Index (NDSI) auf Basis des grünen Lichts (Green) und SWIR definiert:

$$\mathrm{NDSI} = \frac{(\mathrm{Green} - \mathrm{SWIR})}{(\mathrm{Green} + \mathrm{SWIR})}.$$

Der Normalized Difference Built-up Index (NDBI) hingegen erlaubt die Erkennung von bebauten Flächen auf Basis von mittlerem Infrarot (MIR) und NIR:

$$\mathrm{NDBI} = \frac{(\mathrm{MIR} - \mathrm{NIR})}{(\mathrm{MIR} + \mathrm{NIR})}.$$

Alle Indizes produzieren Werte im Bereich von $+1$ bis -1. Höhere Werte deuten dabei auf grüne Vegetation, Wasser, Schnee und Bebauung hin, wohingegen mittlere und niedrige Werte gegen deren Vorliegen sprechen.

1. Implementiert die Indizes und wendet sie auf das Satellitenbild in Abbildung 2 an (s. `landsatBild.zip` im Moodle). Für das Bild stehen euch dazu im `landsatBild.zip` die 7 Bänder des jeweiligen Landsat 5-Satellitenbildes zur Verfügung. Die Bänder sind einzeln als Graustufenbilder gespeichert. Zeigt die vier Ergebnisbilder mit `matplotlib` an.
   Tipp 1: Um eine realistische Darstellung zu erhalten, passt bei `matplotlib.pyplot.imshow` die Parameter `vmin` und `vmax` auf den Wertebereich von NDVI bzw. NDWI an.
   Tipp 2: Schaut in die Folien zu Vorlesung 2, um euch zu Landsat 5-Satellitenbildern zu informieren.
2. Analysiert das Satellitenbild in Abbildung 2 anhand der Indizes und des Echtfarbenbildes. Worum handelt es sich bei den dunklen Flächen im Bild in etwa? Den dunklen Strich oben links könnt ihr ignorieren. Gebt eine kurze Antwort mit Begründung (max. 3 Sätze).

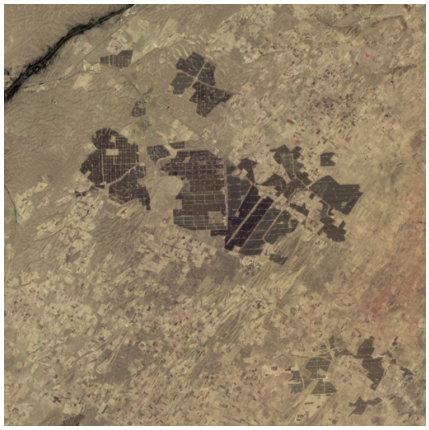


Abbildung 2: Echtfarben-Darstellung eines Landsat 5-Satellitenbildes.

Zusatzaufgabe 5 — Überlaufbehandlung bei Bildern - 5+5+5+5=20 Punkte - Programmieraufgabe
Erlaubte (Sub-)Pakete: `numpy`, `skimage.io`, `skimage.util`, `matplotlib`
In dieser Aufgabe ist die Nutzung von Schleifen nicht erlaubt!
In dieser Aufgabe soll das Bild `einstein.png` aus dem Moodle aufgehellt und der dabei entstehende Überlauf behandelt werden.
Hinweis: Setzt in dieser Aufgaben wie auch bei allen anderen Aufgaben die Parameter `vmin` und `vmax` der Funktion `matplotlib.pyplot.imshow` auf das Minimum bzw. Maximum des Wertebereichs. Andernfalls kommen falsche Darstellungen heraus.

1. Schreibt zunächst eine Funktion, die jedes Pixel des Bildes um 100 Graustufen aufhellt und behandelt den Überlauf per *wrap-around* (s. VL-Folien). Wendet die Funktion auf das Bild an. Zeigt anschließend das Ergebnis an.
2. Ändert nun die Überlaufbehandlung auf *saturation* (s. VL-Folien). Wie verändert sich das Ergebnis?
3. Ändert nun die Überlaufbehandlung auf *re-scaling* (s. VL-Folien). Wie unterscheiden sich das Eingabebild und das Ergebnis? Erklärt das Ergebnis.
4. Schlagt nun eine Formel für die Aufhellung vor, die unter allen Überlaufbehandlungen das Bild aufhellt.